

Praktikum 2 - Anforderungen (Trainingsphase 2/2)

Aufgabe 1 (Beschreibung erweitern)

Wählen Sie sich ein Projekt eines Kommilitonen aus [bitte in Absprache], für das bereits in Praktikum 1 eine Beschreibung erstellt wurde. Lesen Sie sich auch, falls vorhanden, dort vorhandene "Anmerkungen" (siehe Praktikum 1, Aufgabe 4) durch. Kopieren Sie den Beschreibungstext in ein eigenes Dokument und erweitern Sie diese Beschreibung nun um die folgenden, konkreten Informationen:

- **Umgebung:** In welcher Umgebung (physischer Arbeitsplatz und auch technische Systemumgebung, sowohl Hardware wie auch Software) soll die Software eingesetzt werden?
- **Benutzer:** Welche Benutzergruppen/Zielgruppen hat die Software?
- **Funktionen:** Welche Hauptfunktionen hat die Software für ihre Nutzer und was tun diese konkret damit?
- **Beschränkungen:** Wodurch ist Entwicklung oder Einsatz der Software möglicherweise eingeschränkt?
- **Voraussetzungen:** Welche Voraussetzungen benötigt die Software vor der bzw. für die Entwicklung (Hardware, Software, Netzwerk, Vorwissen, Zeit, Personal, etc.)?



Datei ablegen: Legen Sie diese Informationen dann in einer Datei gespeichert (z.B. mit dem Namen "DATUM_PROJEKT_Erweiterungen_BEARBEITER") in Ihrem persönlichen Ordner unter "Übungsaufgaben" ab.

Aufgabe 2 (Funktionale und Nicht-Funktionale Anforderungen formulieren)

Wählen Sie nun erneut ein Projekt eines Kommilitonen aus [bitte in Absprache], entweder dasselbe wie aus der vorigen Aufgabe oder ein anderes, und formulieren Sie nun für die Softwarebeschreibung, unter Berücksichtigung der Ergebnisse aus der vorigen Aufgabe (falls vorhanden), die folgenden Dinge:

- 4 User-Stories
- Insgesamt 8 funktionale Anforderungen:
 - 4 gute (die "Kriterien einer guten SAS" sollen möglichst erfüllt sein: Gehen Sie die Liste der 9 Kriterien für gute Anforderungen aus der Vorlesung vollständig durch und prüfen Sie, ob jedes erfüllt ist)
 - 4 weitere gute, diesmal nach "Easy Approach to Requirements Syntax (EARS)", die Sie um entsprechend notwendige Details (damit die

Anforderung "gut" wird) ergänzen

- Eine Produktqualitätsmatrix nicht-funktionaler Anforderungen (nach IEC/ISO 25010)
- Verwenden Sie 3 verschiedene (selbst gewählte) Metriken um entsprechende nicht-funktionale Anforderungen (gewünschte Systemeigenschaften) der Software zu quantifizieren (d.h. die Eigenschaft soll in Zahlen ausgedrückt messbar-überprüfbar sein!)



Datei ablegen: Speichern Sie diese Formulierungen wie üblich auch wieder in einer Datei ab (z.B. mit Namen "DATUM_PROJEKT_Anforderungen_BEARBEITER") und legen Sie diese in Ihren persönlichen Ordner unter "Übungsaufgaben" ab.



Hinweis: Falls Sie Sich fragen, ob Ihre Anforderung "gut" ist:

- Bitten Sie einen Kommilitonen sich den Anforderungssatz durchzulesen und dazu kritisch Rückmeldung zu geben (Ist etwas unklar, unvollständig oder uneindeutig definiert?)
- Prüfen Sie einzeln die Kriterien für gute Anforderungen (siehe Vorlesung), ob sie erfüllt sind (oder zumindest nicht schwerwiegend gebrochen).

Aufgabe 3 (Schlechte Anforderungen)

- Formulieren Sie nun 4 weitere funktionale Anforderungen, diesmal jedoch schlechte: Jede dieser Anforderungen soll mindestens eines der Kriterien für eine gute SAS brechen.
- Erstellen Sie nun einen Test für ihre Kommilitonen: Mischen Sie dort 4 gute Anforderungen aus der vorigen Aufgabe mit den 4 schlechten Anforderungen aus dieser Aufgabe, so dass nicht erkennbar ist, welche gut und welche schlecht ist.
- Bitten Sie nun einen Kommilitonen sich diesem Test zu unterziehen: Die 4 schlechten Anforderungen sollen gefunden werden und jeweils das gebrochene Kriterium genannt werden.



Datei ablegen: Speichern Sie auch diese Datei wieder unter "Übungsaufgaben" ab, z.B. mit dem Betreff "TestAnforderungen".

Aufgabe 4 (Projektvorstellung)

Wählen Sie Sich nun ein Projekt aus, mit dem Sie Sich in einer der vorigen Aufgaben etwas mehr auseinander gesetzt haben und dessen Beschreibung Sie erweitert haben. Stellen Sie dieses Projekt, basierend auf den zuvor erarbeiteten Erweiterungen, einem oder mehreren Kommilitonen mündlich kurz vor. Bitten Sie auch hier wieder um Rückmeldung dazu: Sowohl positiv ("Was ist gut und sollte beibehalten werden?") als auch konstruktiv ("Was könnte besser gemacht werden?"),

also ob es noch Unklarheiten gibt oder Dinge in der Beschreibung fehlen - bitten Sie um mindestens eine kritische Frage, die weiterführende Details zum Projekt herausfordert.



Datei ablegen: Speichern Sie dieses Fazit mit positiven wie auch negativen Punkten im zugehörigen Projektordner ab.



Hinweis: Hiermit ist Ihre Trainingsphase abgeschlossen. Sie sollten bis jetzt erkannt haben, dass die korrekte, vollständige und "gute" Beschreibung einer Software nicht-trivial ist, sondern Aufmerksamkeit erfordert. Im Folgenden werden Sie Ihre Kenntnisse für ein Projekt einsetzen können, welches Sie im Team nach dem Software Lifecycle entwickeln können. Dabei unterstützt Sie die Plattform Redmine (siehe nächste Aufgabe).

Aufgabe 5 (Projekt migrieren)

Sie sollten mittlerweile für die Projektmanagementplattform "Redmine" freigeschaltet worden sein (<http://werre.ee.hs-owl.de/>). Sie haben dort auch selbst die Möglichkeit Projekte anzulegen. Legen Sie dort ein neues Projekt an. Benennen Sie das Projekt entsprechend eines der zuvor von Ihnen erstellen oder bearbeiteten Projekte und migrieren (herüberkopieren) Sie die zugehörigen Dateien auf die Plattform. Im Projekt selbst können Sie Redmine auch als "Projektmanager" kennenlernen. Probieren Sie in diesem Projekt alle Funktionen aus, die Sie möchten, es dient für Sie als "Trainingsgelände" und wird später nicht weiter genutzt werden. Lassen Sie die Markierung als "Öffentlich" bitte bestehen, damit auch Ihre Kommilitonen automatisch lesenden Zugriff darauf haben und mit daran lernen können. Stöbern Sie gerne auch mal in die Projekte der Kommilitonen herein, Sie werden sehen, dass Sie dort nicht die umfänglichen Rechte haben wie in Ihrem eigenen Projekt. Sie können Ihrem Projekt auch Mitglieder hinzufügen und diese bestimmten Rollen (mit Rechten) zuteilen. Hilfe erhalten Sie auch auf der Redmine-Webseite (<https://www.redmine.org/guide>).

Probieren Sie insbesondere die folgenden Dinge einmal aus:

- (Fiktive) Tickets, Unteraufgaben, Zeitaufwände hinzufügen
- Gantt-Diagramm aktivieren, indem ein Ticket mit Abgabedatum addiert wird
- Mitglieder zum Projekt hinzufügen mit bestimmten Rechten (z.B. "Entwickler")
- Ein Ticket einem Mitglied zuweisen, das dann einen Zeitaufwand dafür einträgt
- Wiki bearbeiten und Dateien ablegen



Hinweis für Fortgeschrittene: Sie können auch, wenn Sie mögen, ein eigenes "Repository" (SVN, GIT [1,2], CVS, usw.) anlegen und in Verbindung mit dem Projekt nutzen (eine direkte technische Anbindung mittels "Konfiguration" in Redmine ist leider aktuell noch nicht möglich). Dieses Repository* müssen Sie gesondert

selbstständig einrichten und verwalten. Beachten Sie jedoch, dass Sie im Laufe des Projekts eine Ticketliste anlegen müssen, die als Ergebnis gilt, welches über Redmine abgenommen werden soll.

* Ein Repository ist ein digitales Archiv, z.B. für die Quelltextdateien (u.A.) eines Software-Projekts. Auf das Archiv können mehrere Nutzer (in manchen Projekten hunderte) Zugriff haben, die an den Dateien arbeiten und Veränderungen implementieren.

[1] <https://git-scm.com/>

[2] Liste von GIT-Anbietern: https://de.wikipedia.org/wiki/Liste_von_Git-GUIs